AWS 클라우드 인프라 설계 및 구현 보고서

교육 콘텐츠 웹 서비스 — 아키텍처 설계부터 운영 자동화까지

프로젝트	교육 콘텐츠 웹 서비스 — AWS 인프라 설계
역할	인프라 아키텍트 (클라우드 아키텍처 설계)
최종 버전	v4.5 (NAT 장애 조치 + 연결성 카나리 자동화)
프로젝트 기간	2025 년 12 월 ~ 2026 년 1 월 (약 2 주)
작성일	2026 년 2 월 21 일
리전	ap-northeast-2 (서울)

1 프로젝트 개요

1.1 배경 및 목표

본 보고서는 교육 콘텐츠 웹 서비스를 위한 AWS 클라우드 인프라 설계 및 구축에 관한 4 인 팀 프로젝트를 기록한 문서입니다. 웹 기능 구현보다 AWS 인프라 설계, 보안, CI/CD 파이프라인 구축, IaC(Infrastructure as Code), 운영 자동화 분야의 실무 역량 검증이 주요 목표였습니다. 애플리케이션 코드는 AI 보조 개발로 최소한으로 구현하여, 팀이 아키텍처 의사 결정과 협업 엔지니어링 프로세스에 집중할 수 있도록 했습니다.

담당 범위: 아키텍처 설계, IaC 초안 작성/명세 제공, 의사 결정 문서화 (실제 구현은 다른 팀원이 수행)

1.2 팀 구성 및 역할 분담

역할	담당 업무
인프라 아키텍트 (설계 리드)	AWS 아키텍처 설계 및 CloudFormation YAML 작성 (설계 산출물)
팀원	인프라 엔지니어 — AWS 스택 배포 및 빌드 단계 트러블슈팅
팀원	보안 엔지니어 — IAM, 네트워크 보안, 접근 제어 설계 및 구현
팀원	앱 개발 & CI/CD 리드 — 애플리케이션 개발, CI/CD 구현, WBS 관리

1.3 기술 스택

구분	내용
클라우드 플랫폼	AWS (ap-northeast-2, 서울)
IaC 도구	AWS CloudFormation (YAML)
애플리케이션 런타임	Python 3.11 / Django / Gunicorn

구분	내용
웹 서버	Nginx (리버스 프록시)
데이터베이스	Amazon RDS MariaDB (Multi-AZ)
CI/CD	GitHub Actions + CodeDeploy (팀 구현) / CodePipeline 설계 제공
시크릿 관리	AWS SSM Parameter Store (SecureString + KMS)
인스턴스 접근	AWS SSM Session Manager (SSH 없는 운영)
OS / 인스턴스	Ubuntu 22.04 LTS / t3.micro

2 아키텍처 의사 결정 과정

인프라 설계는 단번에 완성된 것이 아니라, 초기 설계 → 배포 피드백 → 설계 수정 → 자동화 강화의 연속적인 사이클을 통해 발전했습니다. 이 섹션은 그 변화 과정과 각 핵심 결정의 근거를 기록합니다.

2.1 초기 설계 (v1~v2.1) — 비용 최적화 & 최대 보안

초기 설계 목표

- 비용 최적화: NAT Gateway 완전 제거, VPC 엔드포인트만으로 AWS 서비스 접근
- 보안: 네트워크 레벨에서 프라이빗 서브넷의 모든 인터넷 아웃바운드 차단
- 고가용성: 2-AZ 서브넷 분산 + RDS Multi-AZ
- 재현성: CloudFormation 스택으로 반복 가능한 생성/삭제 사이클

NAT Gateway 제거는 비용 절감뿐 아니라 의도적인 보안 설계 선택이었습니다. 프라이빗 서브넷의 아웃바운드 인터넷 접근을 차단함으로써 잠재적인 C2(명령 및 제어) 통신 경로와 데이터 유출 채널을 제거했습니다. 대신 VPC 엔드포인트를 통한 명시적 허용 목록으로 AWS 서비스 접근을 제한하여 네트워크 레벨의 최소 권한 원칙을 적용했습니다.

2.2 실제 제약 사항 — 배포 피드백

배포 중 발생한 이슈

- ① 인터넷 아웃바운드 차단으로 인한 패키지 다운로드 실패 (apt/pip)
- ② CodeDeploy 에이전트 설치 불안정 (에이전트가 AWS S3 리전 엔드포인트에서 다운로드)
- ③ NAT 없는 환경에서 불안정한 부트스트랩 성공률
- → 실제 AWS 배포 중 인프라 엔지니어의 반복적인 피드백을 통해 식별된 이슈

인프라 엔지니어가 CloudFormation YAML 로 배포를 시도하면서 NAT 없는 환경의 한계가 드러났습니다. 이론적으로는 충분했지만, S3 게이트웨이 엔드포인트와 SSM 인터페이스 엔드포인트가 CodeDeploy 에이전트 부트스트랩 단계에서 필요한 모든 아웃바운드 경로를 완전히 커버하지 못해 간헐적 장애가 발생했습니다. 이 피드백을 바탕으로 '완전 차단'에서 '최소한의 필요 허용'으로 전환하는 설계 수정을 진행했습니다.

2.3 설계 수정 (v3.x~v4.3) — 배포 현실 반영

핵심 수정 원칙

- 'NAT 완전 제거'에서 '단일 NAT(AZ-A) + 엔드포인트 우회'로 전환하여 균형 잡힌 접근 방식 채택
- AWS 서비스 트래픽(S3, SSM)은 여전히 엔드포인트 경유 — NAT 사용 및 노출 최소화
- ElastiCache 제거 → 로컬 Redis 채택 (데모 범위에 맞게 경량화)
- CI/CD 파이프라인을 infrastructure.yaml 에 통합 → 단일 스택으로 전체 환경 재현 가능

단일 NAT Gateway(AZ-A 만)로 최소화하여 비용을 줄이면서 외부 패키지 다운로드의 안정적인 부트스트랩을 복원했습니다. S3 와 SSM 트래픽은 VPC 엔드포인트를 통해 계속 라우팅되어 NAT 노출 인터넷 트래픽을 최소로 유지했습니다.

2.4 최종 설계 개선 (v4.5) — 운영 자동화

단일 NAT Gateway 는 비용 효율적이지만 단일 장애점(SPOF)을 만듭니다. AZ-A 가 다운되면 양쪽 프라이빗 앱 서브넷의 아웃바운드 인터넷 접근이 영향을 받습니다. 이를 완화하기 위해 v4.5 에서 Lambda 기반 자동 장애 조치를 도입했습니다.

v4.5 자동화 — NAT 장애 조치 + 연결성 카나리

- ① 연결성 카나리 Lambda: 프라이빗 서브넷에서 주기적으로 아웃바운드 연결성 테스트 → 사용자 영향 전에 NAT 이슈 사전 감지
- ② CloudWatch 알람: NAT ErrorPortAllocation / PacketsDropCount 또는 카나리 실패 모니터링
- ③ EventBridge: 알람 상태 변경 이벤트 수신 후 장애 조치 Lambda 트리거
- ④ NAT 장애 조치 Lambda: AZ-C 퍼블릭 서브넷에 장애 조치 NAT Gateway 자동 생성 후 프라이빗 앱 라우팅 테이블 기본 경로 업데이트
- ⑤ 자동 롤백: 알람이 OK 상태로 복귀 시 원래 경로(NAT-A) 복원 및 장애 조치 NAT/EIP 삭제

파라미터	기본값	설명
NatGatewayMode	single	single: AZ-A 에만 NAT (비용 최적화) / per-az: AZ 별 NAT (고가용성)
EnableNatAutoFailover	true	Lambda 기반 자동 NAT 장애 조치 활성화
EnableNatConnectivityCanary	true	주기적 아웃바운드 연결성 테스트 Lambda 활성화
CreatePipeline	true	스택에 CodePipeline/CodeBuild CI/CD 리소스 포함

3 설계 버전 이력

배포 피드백이 반영되면서 CloudFormation YAML 의 전체 변화 과정을 아래 표에 정리했습니다.

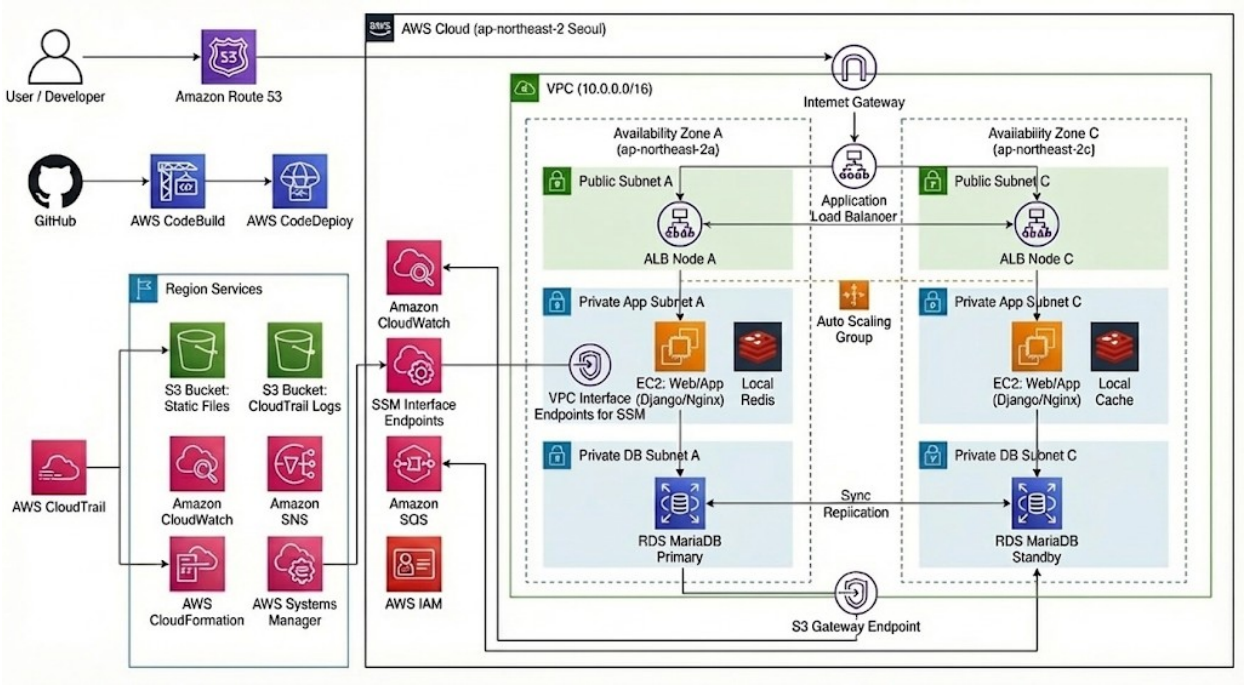
버전	날짜	주요 변경 사항 및 설계 의도
v1	2025.12.23	VPC + ALB + ASG + RDS 기본 구성. NAT 없음 — 엔드포인트 전용 프라이빗 연결 시도
v2.1	2025.12.24	CI/CD 파이프라인을 pipeline.yaml 로 분리, SSM 환경 변수 주입 스크립트 추가
v3.0	2025.12.31	배포 피드백 반영: NAT Gateway(단일) 도입. 골든 AMI 베이킹 방식 채택
v3.1~3.4	2025.12.31	배포 훅을 단계별로 분리(00~04), Gunicorn systemd 유닛 추가, AMI 스크립트 정제

버전	날짜	주요 변경 사항 및 설계 의도
v3.5~3.6	2025.12.31	00_inject_env_from_ssm.sh 추가 (SSM → .env 자동화), Redis AMI 분리
v3.8	2025.12.31	infrastructure.yaml 대폭 확장(27KB), 통합 및 안정화
v4.3	2026.01.05	단일 스택 통합: Nginx 설정 + systemd 유닛 패키징, CI/CD 파이프라인 통합
v4.5 (최종)	2026.01.05	NAT 장애 조치 Lambda + 연결성 카나리 자동화 추가, 운영 자동화 완성

4 최종 아키텍처 상세

4.1 아키텍처 개요

최종 아키텍처(v4.5)는 두 가용 영역(AZ-A, AZ-C)에 걸쳐 퍼블릭 / 프라이빗 앱 / 프라이빗 DB 의 3-Tier 레이아웃으로 구성됩니다. 핵심 설계 성과는 단일 NAT Gateway 를 통한 비용 효율성과 Lambda 기반 장애 조치 자동화를 통한 운영 탄력성의 균형입니다.



▲ *최종AWS 인프라 아키텍처 다이어그램(2-AZ, 3-Tier)*

4.2 네트워크 레이아웃

서브넷	CIDR	AZ	용도
퍼블릭 서브넷 A	10.0.0.0/24	ap-northeast-2a	ALB 노드 A, NAT Gateway A (기본)
퍼블릭 서브넷 C	10.0.1.0/24	ap-northeast-2c	ALB 노드 C, 장애 조치 NAT (자동 생성)
프라이빗 앱 서브넷 A	10.0.10.0/24	ap-northeast-2a	EC2 웹/앱 인스턴스 (Django+Nginx+Redis)
프라이빗 앱 서브넷 C	10.0.11.0/24	ap-northeast-2c	EC2 웹/앱 인스턴스 (Django+Nginx+로컬 캐시)
프라이빗 DB	10.0.20.0/24	ap-northeast-2a	RDS MariaDB Primary

서브넷	CIDR	AZ	용도
서브넷 A			
프라이빗 DB 서브넷 C	10.0.21.0/24	ap-northeast-2c	RDS MariaDB Standby

4.3 보안 그룹 설계

보안 그룹은 최소 권한 원칙에 따라 설계되었으며, 동적 오토스케일링 환경에서 정밀한 접근 제어를 유지하기 위해 IP 범위 대신 보안 그룹 ID 를 소스로 참조합니다.

보안 그룹	인바운드	소스	용도
AlbSG	TCP 80	0.0.0.0/0	인터넷 → ALB
AppSG	TCP 80	AlbSG	ALB → EC2 앱 서버
DbSG	TCP 3306	AppSG	EC2 → RDS MariaDB
VpcEndpointSG	TCP 443	VPC CIDR	EC2 → SSM/S3 인터페이스 엔드포인트
RedisSG	TCP 6379	AppSG	EC2 앱 서버 → 로컬 Redis EC2

4.4 VPC 엔드포인트

인터넷 노출을 최소화하기 위해 모든 AWS 관리형 서비스 접근은 VPC 엔드포인트를 통해 처리합니다:

- S3 게이트웨이 엔드포인트: S3 트래픽(아티팩트, 정적 파일)을 VPC 내부 경로로 라우팅 — 인터넷 노출 없음, 데이터 전송 비용 없음
- SSM 인터페이스 엔드포인트 (ssm, ec2messages, ssmmessages): SSH 없이 Session Manager 로 인스턴스 접근 가능, 운영이 VPC 내부에서 완결

5 주요 컴포넌트

5.1 Application Load Balancer (ALB)

- 퍼블릭 서브넷의 2-AZ 에 걸친 인터넷 페이싱 ALB
- HTTP(80 번 포트) 리스너 → 프라이빗 앱 서브넷 EC2 타겟 그룹으로 포워딩
- 헬스 체크: GET /healthz (HTTP 200~399, 15 초 간격, 임계값: 정상 2 / 비정상 3)
- Nginx 는 /healthz 에 즉시 200 응답하도록 설정 → 앱 시작 전에도 ALB 헬스 체크 통과 가능

5.2 EC2 Auto Scaling Group (ASG)

- 런치 템플릿: Ubuntu 22.04 LTS, t3.micro, Nginx + CodeDeploy 에이전트를 UserData 로 자동 설치
- Desired: 2 / Min: 2 / Max: 4 (2-AZ 에 균등 분산)
- ELB 헬스 체크 연동 (GracePeriod: 180 초)
- IAM 인스턴스 프로파일: SSM Session Manager 접근 + SSM Parameter Store 읽기 + S3 아티팩트 접근

5.3 RDS MariaDB (Multi-AZ)

- 엔진: MariaDB / 인스턴스: db.t3.micro / 스토리지: 20GB gp2
- Multi-AZ: Primary(AZ-A) + Standby(AZ-C), 자동 장애 조치
- 프라이빗 DB 서브넷에 배치 — 퍼블릭 접근 불가
- 모든 연결 정보는 SSM Parameter Store 에 저장 (하드코딩된 자격증명 없음)

5.4 SSM Parameter Store — 환경 변수 관리

SSM 파라미터 경로	타입	용도
/{prefix}/db/host	String	RDS 엔드포인트 — CloudFormation Output 에서 자동 주입
/{prefix}/db/name	String	데이터베이스 이름
/{prefix}/db/user	String	데이터베이스 사용자
/{prefix}/db/password	SecureString	데이터베이스 패스워드 (KMS 암호화)
/{prefix}/django/secret_key	SecureString	Django SECRET_KEY (KMS 암호화)
/{prefix}/django/allowed_hosts	String	Django ALLOWED_HOSTS
/{prefix}/django/debug	String	Django DEBUG 플래그
/{prefix}/app/static_bucket	String	정적 파일 S3 버킷 이름

6 CI/CD 파이프라인

6.1 설계 vs 구현

인프라 아키텍트로서 CloudFormation 스택에 통합된 AWS CodePipeline 을 활용한 CI/CD 파이프라인 명세를 설계했습니다. 팀은 최종적으로 구현 시 GitHub Actions + CodeDeploy 방식을 채택했으며, 두 방법 모두 동일한 CodeDeploy 배포 훅 구조를 공유합니다.

구분	설계 (CloudFormation)	구현 (GitHub Actions)
트리거	GitHub → CodeStar Connection → CodePipeline	GitHub 푸시 → GitHub Actions 워크플로
빌드	CodeBuild (buildspec.yml)	GitHub Actions → AWS CLI → S3 업로드
배포	CodeDeploy (appspec.yml)	CodeDeploy (동일한 appspec.yml)
장점	단일 스택으로 전체 환경 재현 가능	간단한 설정, GitHub 에서 통합 관리

6.2 CodeDeploy 배포 훅

라이프사이클 이벤트	스크립트	수행 작업
ApplicationStop	10_stop_services.sh	실행 중인 Gunicorn 및 Nginx 서비스 중지
BeforeInstall	01_prepare_dirs.sh	배포 디렉토리 생성 및 권한 설정
AfterInstall	02_install_runtime_files.sh	Nginx 설정 및 systemd 유닛 파일 설치
ApplicationStart	03_start_services.sh	SSM 에서 .env 주입 후 Gunicorn 및 Nginx 시작

라이프사이클 이벤트	스크립트	수행 작업
ValidateService	04_validate_service.sh	/healthz 엔드포인트 curl 로 배포 검증

7 관찰 가능성 (Observability)

7.1 CloudWatch 알람

알람	메트릭	임계값	목적
{prefix}-alb-target-5xx	HTTPCode_Target_5XX_Count	≥ 1 / 1 분	애플리케이션 오류 즉시 감지
{prefix}-asg-inservice-low	GroupInServiceInstances	< 1 (2 회 연속)	전체 인스턴스 장애 감지
NatConnectivityAlarm	Lambda 카나리 오류	카나리 실패	선제적 NAT/아웃바운드 이슈 감지

7.2 CloudTrail 감사 로깅

모든 AWS API 호출은 CloudTrail 에 캡처되어 전용 S3 버킷에 저장됩니다. 버킷 정책은 CloudTrail 서비스만 쓰기 접근할 수 있도록 제한합니다. 포괄적인 감사 적용을 위해 멀티 리전 이벤트와 글로벌 서비스 이벤트가 포함됩니다.

7.3 SNS 알림

CloudWatch 알람은 즉각적인 이메일 알림을 위해 SNS 토픽으로 라우팅됩니다. NAT 장애 조치 Lambda 실행 시에도 SNS 알림이 트리거되어 장애 감지, 자동 복구, 복원 상태를 운영자가 완전히 파악할 수 있습니다.

8 설계 비교: 초기 vs 최종

항목	초기 설계 (v1~v2.1)	최종 설계 (v4.5)
NAT Gateway	없음 (비용 & 보안상 완전 제거)	1 개 (AZ-A, 기본 single 모드)
인터넷 아웃바운드	완전 차단 → 엔드포인트 전용 허용 목록	최소 허용(NAT) + 엔드포인트 우회
Redis/캐시	Amazon ElastiCache (검토 중)	로컬 Redis (단순화, 비용 절감)
CI/CD 스택	별도 pipeline YAML	infrastructure.yaml 통합 (단일 스택)
장애 대응	수동 개입	Lambda 장애 조치 + 카나리 자동화
파라미터화	기본 설정	NatGatewayMode, EnableNatAutoFailover 선택 가능
YAML 크기	~18KB	~46KB (대폭 확장)

9 결론 및 기여 요약

9.1 인프라 아키텍트로서의 핵심 기여

- 전체 AWS 인프라 아키텍처 설계: 3-Tier VPC, 서브넷, 보안 그룹, VPC 엔드포인트 구성

- CloudFormation infrastructure.yaml 전체를 처음부터 직접 작성 (v1 → v4.5, 약 2 주간 8 개 버전)
- 실제 배포 피드백을 반영하여 설계를 반복적으로 개선 (NAT 재도입, 배포 스크립트 재구조화)
- v4.5 NAT 장애 조치 Lambda + 연결성 카나리 자동화 설계 및 문서화, 빌드 엔지니어에게 명세 인계
- CodeDeploy 라이프사이클 훅 구조(ApplicationStop → ValidateService) 및 배포 스크립트 명세 설계
- SSM Parameter Store 기반 환경 변수 관리 체계 수립으로 하드코딩된 시크릿 완전 제거
- 발표 Q&A 치트시트 작성 — 모든 아키텍처 결정의 근거 문서화

9.2 핵심 기술 학습

이 프로젝트는 클라우드 아키텍처 개념을 검토 가능한 IaC 명세로 변환하는 실전 경험을 제공했습니다. 가장 중요한 교훈은 최고의 아키텍처는 종이 위에서 가장 우아한 것이 아니라, 실제 배포에서 안정적으로 작동하는 것이라는 점입니다. 이를 위해서는 배포를 실행하는 엔지니어와의 긴밀한 협업과 빠른 피드백 루프가 필수적입니다.

NAT Gateway 결정 과정 — 비용, 신뢰성, 보안의 삼각 트레이드오프 균형 잡기 — 은 실제 클라우드 아키텍처에서 항상 등장하는 과제입니다. 반복적인 개선을 통해 최종 해결책(단일 NAT + 엔드포인트 우회 + Lambda 장애 조치 자동화)에 도달한 과정은 실용적인 시스템 설계의 실증이었습니다.

9.3 향후 개선 방향

- 비용/HA 트레이드오프 개선을 위한 per-az NAT 모드 또는 자체 관리형 NAT 인스턴스 검토
- 배포 안전성 향상을 위한 Blue/Green 배포 전략 도입 (현재 In-Place 방식)
- 관리형 Redis 로 안정성 향상을 위한 Amazon ElastiCache 도입
- CDN 성능 및 웹 애플리케이션 방화벽 보호를 위한 CloudFront + WAF 추가
- 장기 IAM 액세스 키 제거를 위한 역할 기반 임시 자격증명 표준화